김동규 포트폴리오

FindRoot

지도 기반 맛집·코스 공유 웹 서비스

기간 2025.07 ~ 현재 · 개인 프로젝트 (지속 개발·운영, 커밋 113개)

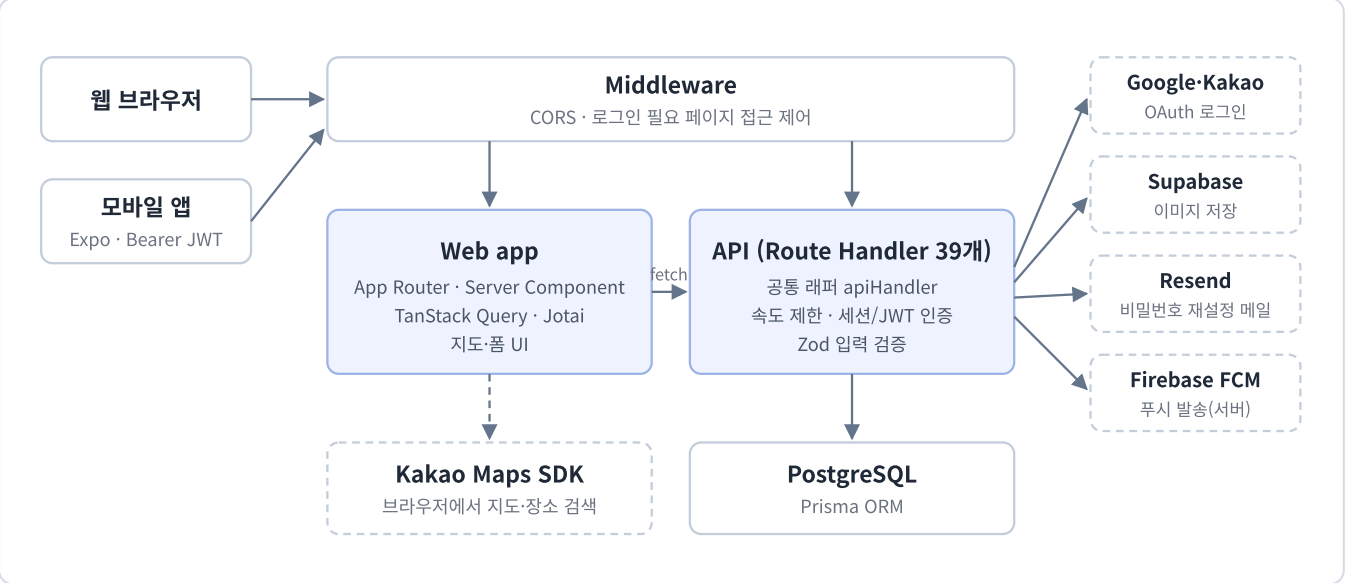
역할 기획 · 프론트엔드 · 백엔드 · 배포 전체

배포 https://www.findroot.kr

GitHub https://github.com/cossgii/findroot

기술 Next.js 15 (App Router) · React 19 · TypeScript · TanStack Query · Jotai · Tailwind CSS · Prisma · PostgreSQL · NextAuth · Jest · Cypress · Storybook

서울 자치구별 맛집과, 그 맛집들을 엮은 루트를 지도 기반으로 등록·공유하는 웹 서비스입니다. 지도 연동, OAuth와 자체 로그인 인증, 장소·루트 등록과 상세 페이지, 마이페이지 등 서비스 전반의 화면과 API(Route Handler 39개)를 설계하고 구현했습니다. 이후 Expo 기반 모바일 앱이 같은 API를 쓸 수 있도록 인증 계층을 JWT 방식까지 확장했습니다.

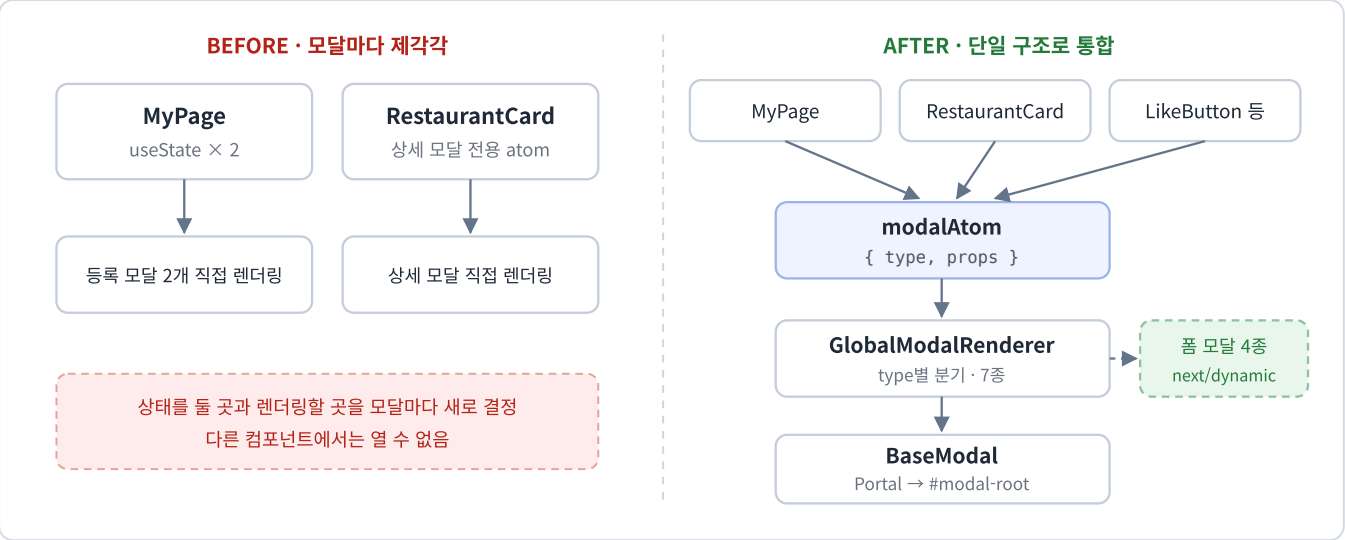


> 웹과 앱의 요청은 모두 같은 API로 들어오고, Route Handler 39개 중 36개는 공통 래퍼(`apiHandler`)를 거쳐 속도 제한, 인증(쿠키 세션 우선, 없으면 Bearer JWT), 입력 검증을 똑같이 적용받습니다. 지도는 브라우저에서 Kakao Maps SDK로 처리하고, 그 밖의 외부 연동(OAuth, 이미지 업로드 URL 발급, 메일, 푸시)은 API 레이어에서 처리합니다.

CASE 01. 전역 모달 렌더링 구조 설계

2025.08 · 관련 커밋 c2af700

> 페이지마다 로컬 state나 전용 atom으로 따로 관리하던 모달을, 단일 atom → GlobalModalRenderer → Portal 구조로 통합했습니다.



문제

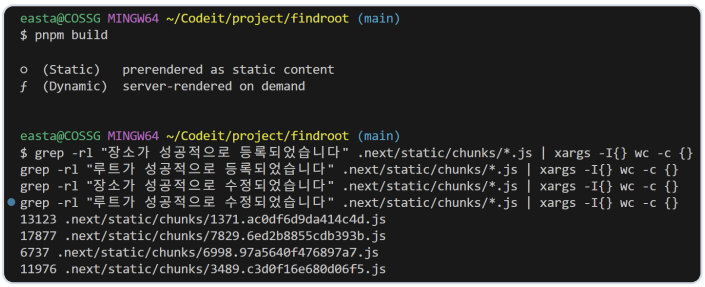
- 모달마다 상태를 두는 방식이 제각각이었습니다. 마이페이지는 장소 등록·루트 등록 모달을 각각 `useState`로, 맛집 상세 모달은 전용 atom으로 관리했습니다.
- 새 모달을 추가할 때마다 상태를 어디에 둘지, 어디서 렌더링할지를 매번 정해야 했습니다.
- 다른 컴포넌트에서 특정 모달을 열 방법이 없어 재사용이 어려웠습니다.

해결

- `{ type, props }` 하나로 모든 모달을 표현하는 단일 Jotai atom을 만들었습니다.
- `GlobalModalRenderer`가 type별로 분기해 7종 모달을 한 곳에서 렌더링하고, `usePathname`으로 라우트가 바뀌면 자동으로 닫히게 했습니다.
- 무거운 등록·수정 폼 모달 4개는 `next/dynamic`으로 분리해 필요할 때만 불러오게 했습니다.

결과

- 어디서든 `setModal({ type, props })` 한 줄로 원하는 모달을 열 수 있습니다.
- 분리한 4개 모달 청크는 총 49,713바이트(약 48.5KB)이고, 초기 페이지 로드에 포함되지 않습니다.
- 이후 추가한 로그인 유도·세션 만료 안내 모달은 타입 하나만 늘려서, 좋아요·팔로우·루트 상세 등 6개 컴포넌트에서 재사용하고 있습니다.



프로덕션 빌드 후 분리된 모달 청크 4개의 크기를 직접 확인한 결과

CASE 02. 로그인 계정 열거 취약점 발견 및 수정

2026.05 ~ 2026.10 · 관련 커밋 9ec8844, 867b1f6, dd614fa, b9c402d

> 소셜 계정으로 가입된 아이디로 일반 로그인을 시도하면 응답에 provider명이 담겨, 존재하지 않는 계정의 응답과 비교하는 것만으로 계정 존재 여부와 가입 경로를 추정할 수 있었습니다. 한 번 고친 문제가 테스트 때문에 되돌아간 사실까지 찾아내 원인부터 다시 고쳤습니다.

로그인 실패 상황	수정 전 2025.07~	1차 수정 2026.05.01	회귀 2026.05.27	재수정 2026.09.14~
없는 아이디	가입되지 않은 아이디입니다.	공통 메시지	공통 메시지	공통 메시지
비밀번호 오류	비밀번호가 일치하지 않습니다.	공통 메시지	공통 메시지	공통 메시지
소셜 가입 아이디	이 아이디는 GOOGLE 계정으로 가입되었습니다.	소셜 계정으로 가입되었습니다. 소셜 로그인을 이용해 주세요.	이 아이디는 GOOGLE 계정으로 가입되었습니다.	공통 메시지

공통 메시지: "아이디 또는 비밀번호가 올바르지 않습니다." 붉은 칸은 응답만으로 계정 존재 여부를 알 수 있는 상태입니다.

문제

- 비밀번호가 없는 계정, 즉 소셜 로그인으로 가입한 계정으로 일반 로그인을 시도하면 응답에 provider명을 그대로 담아 돌려줬습니다.
- 1차 수정에서 없는 아이디와 비밀번호 오류의 메시지는 통일했지만, 소셜 계정에는 여전히 다른 메시지를 돌려줘 계정 존재 여부는 계속 드러났습니다.
- 이후 테스트를 대량으로 추가한 커밋에서 로직이 다시 provider 노출 방식으로 되돌아갔습니다. 기존 테스트가 provider 메시지를 기대값으로 고정하고 있었고, 테스트를 통과시키는 과정에서 잘못된 동작이 복구된 것입니다.

해결

- 계정 존재 여부·가입 방식과 무관하게 모든 인증 실패에 같은 메시지를 반환하도록 통일하고, provider를 확인하려고 하던 추가 DB 조회도 없앴습니다.
- 회귀의 원인이 된 테스트를 "provider 종류와 무관하게 항상 같은 메시지를 반환하고, provider 조회 자체를 하지 않는다"를 검증하도록 다시 작성했습니다.
- 메시지 외의 경로도 이어서 막았습니다. 없는 계정도 더미 해시와 비교해 응답 시간 차이를 줄이고, 웹·앱 로그인 모두에 IP 기준 시도 횟수 제한을 걸고, 비로그인 사용자 조회 API에서 이메일을 제거했습니다.

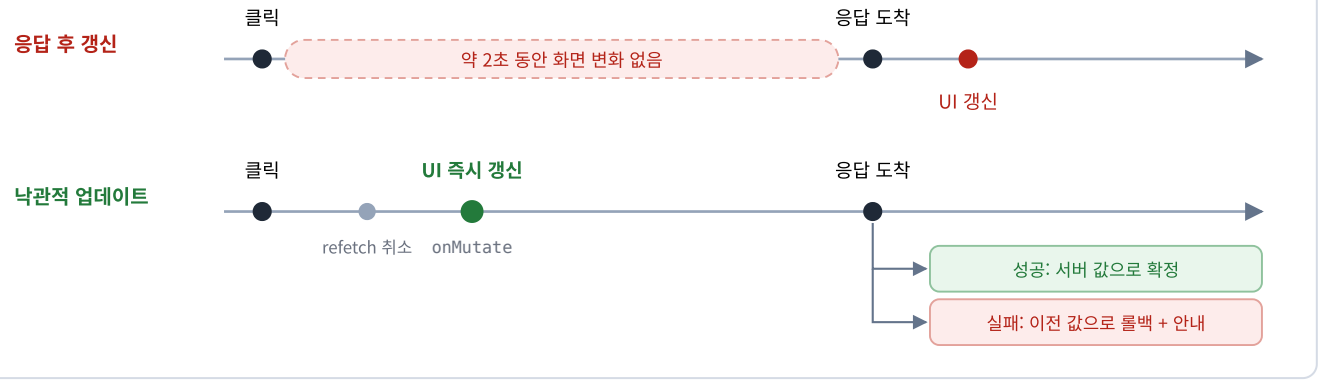
결과

- 로그인 응답의 메시지와 처리 시간만으로는 계정이 있는지, 어떤 방식으로 가입했는지 구분하기 어려워졌습니다.
- 같은 문제가 다시 생기면 단위 테스트가 실패합니다.
- 테스트가 잘못된 동작을 기대값으로 고정하면 오히려 회귀를 만든다는 것을 배웠습니다. 테스트는 현재 동작이 아니라 지켜야 할 규칙을 기준으로 작성해야 합니다.

CASE 03. React Query 낙관적 업데이트로 체감 지연 제거

좋아요·팔로우 · 관련 파일 useLike.ts, FollowButton.tsx

> 서버 응답을 기다리지 않고 `onMutate` 에서 캐시를 먼저 갱신하고, 실패했을 때만 `onError` 로 되돌립니다.



문제

- 배포 환경에서 좋아요 요청은 응답까지 실측 약 2초가 걸립니다. 응답을 받은 뒤에 화면을 바꾸면, 사용자는 그동안 버튼이 눌렸는지조차 알 수 없습니다.
- 같은 데이터를 보여주는 다른 화면(목록, 상세, 마이페이지)에도 변화가 함께 반영돼야 했습니다.
- 요청이 실패했을 때 화면과 서버 상태가 어긋난 채로 남으면 안 됐습니다.

likes	201	fetch	710-c69a7b5d00917d2	0.1 kB	2.07 초
info?routeId=cmqgrdetj000004l4ughnj9...	200	fetch	710-c69a7b5d00917d2	0.1 kB	2.06 초
locations?page=1&limit=10&sort=recent	200	fetch	page-49c3ae8d73020c	4.0 kB	2.12 초

배포 환경에서 측정한 좋아요(likes) 요청의 응답 시간

해결

- `onMutate` 에서 `queryClient.cancelQueries()` 로 진행 중인 같은 쿼리의 refetch를 먼저 취소한 뒤, 캐시를 미리 갱신했습니다.
- 요청이 실패하면 `onError` 에서 이전 상태로 되돌리고 토스트로 알렸습니다.
- 성공하면 서버가 돌려준 최종 값으로 캐시를 맞추고, 좋아요 목록처럼 연관된 쿼리를 함께 무효화했습니다. 팔로우 버튼에도 같은 패턴을 적용했습니다.

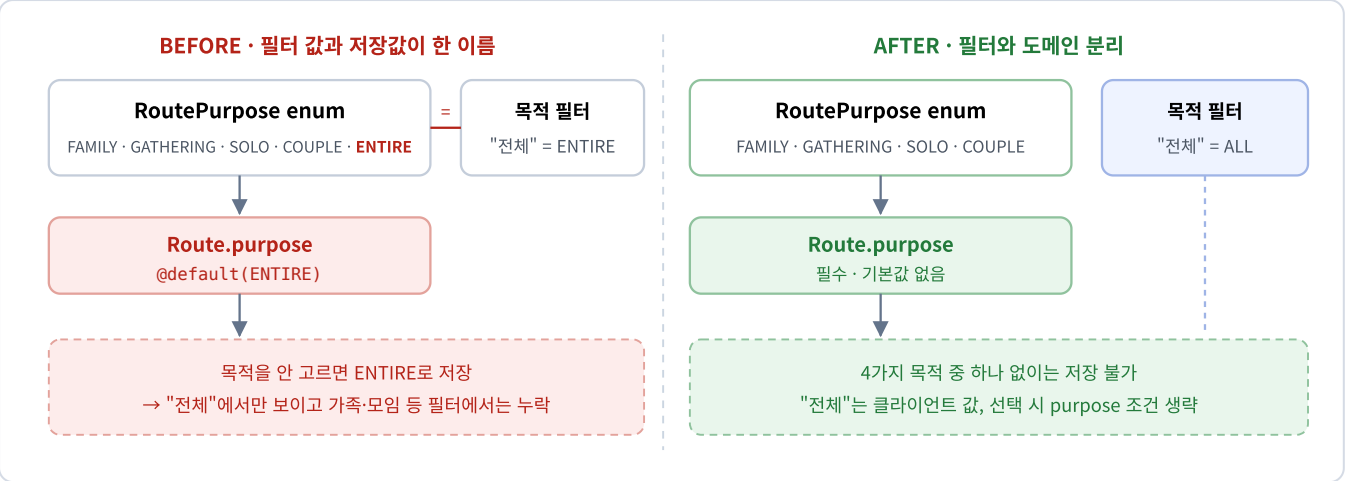
결과

- 서버 응답 시간(약 2초)과 상관없이 클릭하는 즉시 화면이 반응합니다.
- refetch를 먼저 취소해 두어, 늦게 도착한 이전 응답이 낙관적으로 바꾼 값을 덮어쓰지 않습니다.
- 요청이 실패하면 롤백되는지를 Cypress E2E 테스트(서버 500 응답을 가정)로 검증하고 있습니다.

CASE 04. 필터링 시 데이터가 누락되던 구조적 결함 해결

2026.06 · 관련 커밋 59a4934, c3c514e

> 필터에서만 쓰여야 할 "전체" 개념이 DB enum 값(ENTIRE)이 되어 기본값까지 맡으면서, 목적을 고르지 않고 만든 루트가 개별 목적 필터에서 보이지 않았습니다.



문제

- 루트 목적 기능을 도입하면서(2025.12) "전체"를 `RoutePurpose` enum의 `ENTIRE` 값으로 만들고, `Route.purpose`의 기본값으로 지정했습니다.
- 등록 폼은 선택지에서 ENTIRE를 빼 두었지만 초기값은 ENTIRE였습니다. 그래서 화면에는 "루트 목적을 선택하세요"가 그대로 보이는데도 등록이 가능했고, 그 루트는 ENTIRE로 저장됐습니다.
- 목적 필터는 ENTIRE를 "조건 없음"으로 해석했기 때문에, ENTIRE로 저장된 루트는 "전체"에서만 보이고 가족·모임 같은 개별 목적 필터에서는 빠졌습니다.
- 같은 "전체"라는 이름이 UI 선택지와 저장값을 동시에 뜻해서, 목적이 없는 루트인지 모든 목적에 맞는 루트인지 구분할 수 없었습니다.

해결

- `RoutePurpose` enum에서 `ENTIRE`를 제거하고, `Route.purpose`를 기본값 없는 필수 필드로 바꿨습니다.
- 필터의 "전체"는 클라이언트 전용 값(`ALL`)으로 분리해, 선택하면 purpose 조건을 아예 걸지 않도록 했습니다.

결과

- 목적 없이 의미 없는 값이 저장되는 경로 자체가 없어졌습니다.
- 필터(UI 상태)와 저장값(도메인 모델)의 경계가 분명해졌습니다.

```
easta@COSSG MINGW64 ~/Codeit/project/findroot (main)
$ grep -A6 "enum RoutePurpose" prisma/schema.prisma

grep -B2 -A2 "purpose\s*RoutePurpose" prisma/schema.prisma
enum RoutePurpose {
  FAMILY
  GATHERING
  SOLO
  COUPLE
}

  creatorId        String       @map("creator_id")
  districtId       String?      @map("district_id")
  purpose          RoutePurpose
  isRepresentative Boolean      @default(false) @map("is_representative")
  likes            Like[]
```

수정 후 스키마: ENTIRE가 제거된 RoutePurpose와 기본값이 없는 purpose 필드

그 밖에 구현한 것

코드는 GitHub 저장소에서 확인할 수 있습니다.

- **공통 API 래퍼(apiHandler)**: 속도 제한, 인증(쿠키 세션 → Bearer JWT), Zod 입력 검증, 에러 응답 형식을 Route Handler마다 반복하지 않도록 한곳에 모았습니다.
- **모바일 앱 연동**: Expo 앱이 `/api/auth/app-login`으로 JWT를 발급받아 웹과 같은 API를 쓰도록 인증 계층을 확장했습니다.
- **공통 컴포넌트 문서화**: Button, Input, Dropdown 등 공통 컴포넌트 12개의 Storybook 스토리를 작성하고, Button·Input 등은 CVA로 variant를 정리했습니다. a11y 애드온으로 접근성 점검 환경도 갖췄습니다.
- **비밀번호 재설정**: selector/validator로 분리한 1시간 유효 토큰을 사용하고, 계정 존재 여부와 무관하게 같은 응답을 돌려주며, 토큰을 이메일이 아닌 사용자 ID에 연결했습니다.
- **테스트**: Jest 단위·API 테스트와 Cypress E2E 테스트(로그인, 회원가입, 좋아요, 루트 생성, 댓글, 팔로잉 무한 스크롤)를 작성했습니다.
- **공유 이미지**: 루트를 공유할 때 보이는 OG 이미지를 Edge 런타임에서 경유지 정보로 동적 생성합니다.